

**Homework 1**

**Question 1.** Briefly define the following terms in context of computer hardware.

- 1) Register
- 2) Memory
- 3) Disk

**Question 2.** What is the smallest addressable unit of memory in most modern computers?

**Question 3.** Give the number of bytes in the following memory units as either 2 or 10 raised to an appropriate power. For example, a Kibibyte (KiB) is  $2^{10}$  bytes while a Kilobyte (KB) is  $10^3$  bytes.

- 1) Mebibyte (MiB)
- 2) Megabyte (MB)
- 3) Gibibyte (GiB)
- 4) Gigabyte (GB)
- 5) Tebibyte (TiB)
- 6) Terabyte (TB)
- 7) Pebibyte (PiB)
- 8) Petabyte (PB)

Why is there a need for two byte-prefixed unit systems?

**Question 4.** Java class in listing 1 prints the maximum numerical value each Java primitive type can store. Explain how these are calculated, i. e., their connection to register size and numerical sign.

---

```

1  public class Constants {
2      public static void main(String[] args) {
3          System.out.printf("byte    %25d\n", Byte.MAX_VALUE);
4          System.out.printf("short   %25d\n", Short.MAX_VALUE);
5          System.out.printf("char    %25d\n", (int)Character.MAX_VALUE);
6          System.out.printf("int     %25d\n", Integer.MAX_VALUE);
7          System.out.printf("long    %25d\n", Long.MAX_VALUE);
8          System.out.printf("float   %25f\n", Float.MAX_VALUE);
9          System.out.printf("double  %25f\n", Double.MAX_VALUE);
10     }
11 }
```

---

LISTING 1—Java class to print maximum values of primitive types.

**Question 5.** Write a Java program that prints out one line of text to the console. It can be anything but “Hello World!” What did you print? Store the code in a file called Anything.java.

**Question 6.** Write a Java program that populates an array of size  $n$  with the first  $n$  Fibonacci numbers. The program should print out the array as shown in figure 1. Here  $n$  should be the first command line argument. You may do it anyway you like but one and arguably the most elegant way to do it is recursively as shown in the listing 2. What is the name of the implicit call structure that is used in listing 2? Hint: Stack Overflow. Store the code in a file called Fibonacci.java.

---

```

1  public static int fibonacci(int i, int[] arr) {
2      if (i < 2)
3          return arr[i] = i;
4      return arr[i] = fibonacci(i - 1, arr) + fibonacci(i - 2, arr);
5  }
```

---

LISTING 2—Java function to generate the Fibonacci sequence.

**Question 7.** Using the [Sieve of Eratosthenes](#), populate a boolean array of size  $n$  (Java booleans initialise to false) marking all the indices that are Composite numbers to true. Here  $n$  should be the first command line argument. The indices remaining false at the end should be Prime numbers. Store the code in a file called `Eratosthenes.java`.

- 1) For debugging, have your program print all the prime numbers less than a 100. You should get the following: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.
- 2) The program should print out at most the five largest prime numbers it computed and the time (seconds) it took to compute all the primes less than  $n$ . Here is a way to compute seconds taken by a function call `eratosthenes(toSieve)`.

---

```

1  double startTime = System.nanoTime();
2  eratosthenes(toSieve);
3  double duration = System.nanoTime() - startTime;
4  duration = duration / Math.pow(10, 9);

```

---

- 3) With your program, calculate how long does it take (in seconds) to compute all the 30 bit prime numbers. These are all primes less than  $n = 2^{30} = 1073741824$ .
- 4) Can your implementation of the Sieve of Eratosthenes compute all the 32 bit prime numbers? If yes, give the time it takes or if it can not, then why not?

**Question 8.** Read all bytes in the file `half_gaps.bin`. You may use the function in code listing 3.

---

```

1  public static byte[] getFileBytes(String path) {
2      byte[] bytes = null;
3      try {
4          bytes = Files.readAllBytes(Paths.get(path));
5      } catch (IOException e) {
6          e.printStackTrace();
7      }
8      return bytes;
9  }

```

---

LISTING 3—Java function to read in a file's (signed) bytes.

The function in code listing 3 reads in signed bytes. While this maybe suitable for some binary arrangements, we want the bytes to be unsigned. One way to achieve this is to just loop and use `Byte.toUnsignedInt(byte x)` as seen in listing 4.

---

```

1  byte[] bytes = getFileBytes("../media/half_gaps.bin");
2  int[] gaps = new int[bytes.length];
3  for (int i = 0; i < bytes.length; i++)
4      gaps[i] = Byte.toUnsignedInt(bytes[i]);

```

---

LISTING 4—Converting Java signed bytes to unsigned longs.

Compute the array of integers' cumulative sum, i. e.,

$$\left\{ x_i \in \text{cumsum}(x) : x_i = \sum_{k=1}^i x_k \right\}$$

Now multiply each of the sums with 2 and then add a 3.

$$\left\{ x_i \in \text{cumsum}(x) : y_i = 2x_i + 3 = 2 \left( \sum_{k=1}^i x_k \right) + 3 \right\}$$

- 1) Print out the first fifteen and the last five elements of this final array.
- 2) Time this program (the reading of bytes, the cumulative sum computation and the doubling with adding a three) and print the result in seconds.
- 3) Do you recognise the printed numbers? What will these be if we further prepended a 2 and a 3 to them?

Store the code in a file called `Primes.java`.

**Question 9.** Accumulate the approximate probability that an integer  $2 \leq x \leq 2^{31} - 1$  is prime. You can do this by generating random numbers between 2 and  $2^{31} - 1$  within a big enough loop and check if the number is prime (this is known as a primality test) by searching for it in the array of prime numbers we constructed in question 8. The main loop is shown in listing 5. You need to implement the linear search and the binary search and uncomment each, one at a time to report the times in seconds taken by each type of search.

---

```

1      double start = System.nanoTime();
2      double favourable = 0;
3      for (int i = 1; i <= 1000000; i++) {
4          long x = getRandomInt(2, Integer.MAX_VALUE);
5          // favourable += linearSearch(x, primes) ? 1.0 : 0.0;
6          favourable += binarySearch(x, primes, 0, primes.length - 1) ? 1.0 : 0.0;
7          System.out.println(favourable / i);
8      }
9      double duration = (System.nanoTime() - start) / Math.pow(10, 9);
10     System.out.printf("Time taken: %.2f seconds\n", duration);

```

---

LISTING 5—Linear and binary search for a prime number.

Does the printed number converge? Store the code in a file called `Search.java`.

**Question 10.** Break the Affine cipher. Your professor encrypted a plain text file called `plain.txt` using the program given in listing 6. Store the code in a file called `Decipher.java`.

He then redirected the output to a cipher file called `cipher.txt`.

- 1) Use the cipher text file and the code in listing 6 to recover the plain text. *Hint:*  $7^{-1} = 55 \pmod{2^7}$ .
- 2) What should the  $2^7$  tell you about the text encoding of the original plain text file?

---

```

1      import java.io.IOException;
2      import java.nio.file.Files;
3      import java.nio.file.Paths;
4
5      public class Decipher {
6          public static void main(String[] args) {
7              int m = (int) Math.pow(2, 7);
8              char[] cipher = new char[m];
9              for (int i = 0; i < m; i++)
10                 cipher[i] = (char) mod(7 * i + 3, m);
11              String plain = read("../media/plain.txt");
12              for (int i = 0; i < plain.length(); i++)
13                 System.out.print(cipher[plain.charAt(i)]);
14          }
15          public static String read(String path) {
16              String ret = null;
17              try {
18                  ret = new String(Files.readAllBytes(Paths.get(path)));
19              } catch (IOException e) {
20                  e.printStackTrace();
21              }
22              return ret;
23          }
24          public static int mod(int x, int n) {
25              return ((x % n) + n) % n;
26          }
27      }

```

---

LISTING 6—An Affine cipher in Java.

## 10.1. EXAMPLE EXECUTIONS

Figure 1 shows how the output of the code for the files `Fibonacci.java` and `Eratosthenes.java` should look like on the standard out. All your programs must compile/run from the command line using `javac` and `java` commands, e. g.,

```
javac Program.java
java Program
```

```

tfn@othelo [ code ] % javac Anything.java
tfn@othelo [ code ] % java Anything
Slamalikum!
tfn@othelo [ code ] % javac Fibonacci.java
tfn@othelo [ code ] % java Fibonacci
Please enter n.
tfn@othelo [ code ] % java Fibonacci 42
      0      1      1      2      3      5      8
     13     21     34     55     89    144    233
    377    610    987    1597    2584    4181    6765
   10946   17711   28657   46368   75025   121393   196418
  317811  514229  832040  1346269  2178309  3524578  5702887
 9227465 14930352 24157817 39088169 63245986 102334155 165580141
tfn@othelo [ code ] % javac Eratosthenes.java
tfn@othelo [ code ] % java Eratosthenes
Please enter n.
tfn@othelo [ code ] % java Eratosthenes 1073741824
Time taken: 5.21 seconds
1073741789
1073741783
1073741741
1073741723
1073741719
tfn@othelo [ code ] % 

```

FIGURE 1—Example execution of the code for the first three questions.

## 10.2. SUBMISSION INSTRUCTIONS

- Submit `Anything.java`, `Fibonacci.java`, `Eratosthenes.java`, `Primes.java`, `Search.java`, `Decipher.java` and `sol.pdf` at the online classroom.
- The files `Anything.java`, `Fibonacci.java`, `Eratosthenes.java`, `Primes.java`, `Search.java`, `Decipher.java` should contain the Java source code for the relevant questions. Do not turn in the dot class files.
- The PDF file `sol.pdf` should contain written answers to questions as well as a screenshot similar to the one in figure 1 that demonstrates your code being compiled and ran.